

```
Copyright 1991-1994, 1997, 1998, 2000, 2004, 2006 Free Software
Foundation, Inc.
This is free software with ABSOLUTELY NO WARRANTY.
For details type 'warranty'.
2 + 2 * 3 - 5 + 21 / 4 * 6 # A basic maths statement
33
(2 ^ 2) ^ 3 # Another one, with power & brackets
64
^D
```

Yes, you guessed right. # starts a comment, as it does for the shell, or like // in C++. For a multi-line comment, you may use /* */ as in C/C++. You may want that, just in case you are writing a complete program in *bc*. To do that, put your math statements (each one on a line by itself) in a file, say in *prog.bc*, as follows:

```
2 + 2 * 3 - 5 + 21 / 4 * 6 # A basic maths statement
(2 ^ 2) ^ 3 # Another one with power & bracket
quit # This will complete the program and not wait for more input
```

And then execute (yes, I do mean ‘execute’; you do not need to compile it) as follows:

```
$ bc prog.bc
bc 1.06.95
Copyright 1991-1994, 1997, 1998, 2000, 2004, 2006 Free Software
Foundation, Inc.
This is free software with ABSOLUTELY NO WARRANTY.
For details type 'warranty'.
33
64
```

Ah! You have got the welcome message from *bc*, and then your results. To avoid the welcome message, add the *-q* option to the command-line, as shown below:

```
$ bc -q prog.bc
33
64
```

You may also try out the difference in the output of the same program with a *-l* option, i.e., with real numbers. Then, / would be treated as a complete division, not just a quotient provider. Here's what you would get:

```
$ bc -ql prog.bc
34.50000000000000000000000000000000
64
```

Programming with *bc*

As soon as programming is mentioned, the first thing people think of is variables. Yes, so they are there. Variable names must start with a small letter and may contain

numbers or small letters of the alphabets. Yes, you read it right, only small letters (a,b,c,...,z). This is because capital letters (A,B,C,...) are used to represent numbers in other bases greater than 10. *bc* supports various bases from 2 to 16, and two variables associated with them: 1) *ibase*: defines the base for input; and 2) *obase*: defines the base for output. By default, both are set to 10, as in our day-to-day math, but can be modified, for fancier base conversions. Here's a snippet:

```
$ bc -ql
ibase # Show the current input base
10
obase # Show the current output base
10
obase=16 # Set the output base to 16
100 # Input in base 10; Output should be in base 16
6C
obase=10 # Set the output base back to 10
ibase=16 # Set the input base to 16
11 # Input now in base 16; Output should be in base 10
17
ibase=10 # Set the input base to 16. 10 is 16 in input base 16.
ibase=A # Set the input base to 10.
obase=2 # Set the output base to 2, i.e. binary
x = 2 * 5 - 1 # Set the variable x to 9 (input base 10)
x # Display the value of x (in output base 2)
1001
x * 2 # This should display 18 in base 2, but x is still 9
10010
obase=10
x++ # Post incr: Display the current value of x and then incr it
9
x # Display the incremented value
10
--x # Pre decrement: Decrement x and then display its value
9
^D
```

From the demo shown above, you might have already observed that there is nothing like declaring the variable type; just assign them using = and then use them. Moreover, *bc* also has basic conditional and loop constructs: *if*, *for*, *while*, *break*, and *continue*. And along with those are the usual C-like relational (<, <=, >, >=, ==, !=), logical (!, ||, &&), and operation-assignment (-=, +=, *=, /=, %=, ^=) operators. A note of caution: their precedence and associativity rules may not be as in C. If you do not understand that, forget about it—just make sure to use brackets for whatever operations you want to occur first. Here goes two simple programs to demonstrate this point: 1) Computing the sum of the first *n* numbers (*sum.bc*); 2) Computing the product of the first *n* numbers, i.e. factorial of *n* (*factorial.bc*):